Guion de Defensa — Informe de Pasantía

Título: Diseño e implementación de un sistema IoT con Microcontroladores para el Monitoreo Remoto de Equipos Fiscales

Autora: Lorena Del Valle Rojas Chams El Dein — C.I. 19-10469 — Ingeniería Electrónica, Universidad Simón Bolívar
Institución: The Factory HKA
Tutor académico: Prof. Miguel Díaz
Tutores industriales: Ing. Pablo Arguinzones y Ing. Luis Muñoz
Formato: 23 diapositivas · ∼3.300 palabras habladas · **objetivo 30–32 min** + ronda de preguntas

> **Versión condensada (30–32 min).** La versión extensa quedó respaldada en `Speech_Defensa_Pasantias_r`. Este guion ya incorpora las correcciones del libro v2: el enlace local es **síncrono (100 kHz)**, distinto del **RS-232/UART asíncrono** inicial; el conflicto de concurrencia es un **acceso concurrente al bus compartido**, arbitrado con un **mutex de software de FreeRTOS**; el objetivo del servidor MQTT figura como **parcialmente logrado**; y el **Bloque 3 está reenfocado a resultados**.

Cómo usar este guion

- El texto entre **negrita y corchetes** son acotaciones para la ponente (`[PAUSA]`, `[SEÑALAR]`, `[MOSTRAR]`). **No se leen en voz alta.**
- El discurso va en **primera persona del plural** para toda la exposición técnica; el singular se reserva para la presentación personal, los agradecimientos y la reflexión final.
- Los tiempos son acumulativos, calculados a ∼115 palabras por minuto (tu ritmo medido). Si en el ensayo vas más rápido, tendrás margen; si te atascas, recorta la diapositiva 9.
- Las frases de transición cierran cada bloque: son clave para la continuidad.

Mapa de diapositivas (23 en total)

#	Diapositiva	Bloque
1	Portada: título, autora, tutores, institución	1
2	Contexto: The Factory HKA y el parque de impresoras fiscales	1
3	Planteamiento del problema	1
4	Objetivos: general y específicos	1
5	Arquitectura IIoT de tres niveles (Edge / Gateway / Cloud)	2
6	Nivel Edge: captura del *Status S1* por interfaz síncrona	2
7	Nivel Gateway: ESP32, ESP-IDF y kernel concurrente (FreeRTOS)	2
8	Nivel Cloud: MQTT v5.0 / TLS 1.2 y Django + HTMX	2
9	Justificación tecnológica: Arduino vs. ESP-IDF · HTTP vs. MQTT	2
10	Metodología de validación: banco Hardware-in-the-Loop (HIL)	3
11	**Resultado 1** — Captura íntegra e integridad de la telemetría	3
12	**Resultado 2** — Robustez e inmunidad del bus local	3
13	**Resultado 3** — Eficiencia de red: ahorro del 80,2 %	3
14	**Resultado 4** — Resiliencia: cero pérdida de datos	3
15	**Resultado 5** — Latencia End-to-End (T_{E2E})	3
16	**Resultado 6** — Actualización remota OTA de doble objetivo	3
17	**Resultado 7** — Rendimiento global: memoria, energía y trazabilidad	3
18	**Resultado 8** — Comparativa: estado previo vs. estado actual	3
19	Verificación de cumplimiento de objetivos	4
20	Aporte y beneficio para la organización	4
21	Limitaciones encontradas y trabajo futuro	4
22	Escalabilidad y recomendaciones	4
23	Cierre, agradecimientos y paso a preguntas	5

BLOQUE 1 — Introducción y Marco del Problema

Tiempo estimado: 3,6 min (acumulado 0:00 – 3:36) · Diapositivas 1 a 4

Diapositiva 1 — Portada (0:00 – 0:35)

[Diapositiva en pantalla. Habla pausado, mirando al jurado.]

Primero que nada, buenos días. Nos gustaría comenzar extendiéndoles un cordial saludo a todos los presentes y presentándonos: mi nombre es Lorena Rojas. El día de hoy estamos aquí para presentarles la defensa de mi Proyecto de Pasantía, titulado *Diseño e implementación de un sistema IoT con Microcontroladores para el Monitoreo Remoto de Equipos Fiscales.*

Este trabajo fue desarrollado en la empresa The Factory HKA, bajo la tutoría académica del profesor Miguel Díaz y la tutoría industrial del ingeniero Pablo Arguinzones, a quienes agradezco desde ya su acompañamiento durante todo el proceso. [**PAUSA BREVE**]

Diapositiva 2 — Contexto (0:35 – 1:29)

[SEÑALAR la imagen del parque de impresoras o el logo de HKA.]

The Factory HKA es una empresa líder en el sector de equipos fiscales en Venezuela. Su actividad gira en torno a las impresoras fiscales: los dispositivos que, por normativa, registran y certifican cada transacción comercial de un contribuyente.

Históricamente, la fiscalización ha dependido de equipos que operan punto a punto, sobre el estándar serial RS-232, aislados de las redes corporativas. En la práctica, la única forma de diagnosticar o auditar una impresora es la conexión física de un técnico, equipo por equipo. Por eso la empresa ha iniciado una transición hacia la digitalización de estos procesos, y ese es el contexto de la pasantía.

Diapositiva 3 — Planteamiento del problema (1:29 – 2:29)

[PAUSA. Mira al jurado.]

El laboratorio de desarrollo y la unidad de soporte de The Factory HKA enfrentan limitaciones operativas concretas: el monitoreo de los estados de la impresora —por ejemplo, el registro *Status S1*— depende de la presencia física del técnico o de conexiones seriales punto a punto.

Esto genera tres consecuencias. Primero, dificulta la detección temprana de errores críticos, como *memoria fiscal llena* o fallos mecánicos, que pueden interrumpir la actividad comercial de forma imprevista. Segundo, la visualización es fragmentada: los técnicos observan un equipo a la vez, sin visión global del parque. Y tercero, no existe un historial de auditoría consultable.

[PAUSA] En síntesis: **equipos fiscales con datos valiosos, pero aislados, sin telemetría centralizada y sin trazabilidad.**

Diapositiva 4 — Objetivos (2:29 – 3:36)

[SEÑALAR el objetivo general.]

Frente a esto, el objetivo general fue: **diseñar, implementar y validar un sistema basado en microcontroladores ESP32 que permita el monitoreo remoto y en tiempo real de los datos operativos de equipos fiscales, garantizando la confiabilidad y la robustez del sistema de comunicación.**

[SEÑALAR los objetivos específicos. Agrupados.]

Los ocho objetivos específicos se agrupan en tres frentes:

- **Diseño e integración:** definir la arquitectura del sistema IoT y desarrollar el firmware embebido, integrándolo con las interfaces del equipo fiscal.
- **Comunicación y visualización:** implementar la mensajería MQTT, la capa de recepción y distribución de los datos, y el dashboard web en tiempo real.
- **Validación y documentación:** evaluar confiabilidad, latencia y robustez ante fallos, y documentar todo el proceso.

[TRANSICIÓN al Bloque 2] Con el problema y los objetivos claros, pasamos a mostrarles cómo diseñamos la solución.

BLOQUE 2 — Arquitectura y Diseño de la Solución

Tiempo estimado: 7,2 min (acumulado 3:36 – 10:50) · Diapositivas 5 a 9

Diapositiva 5 — Arquitectura IIoT de tres niveles (3:36 – 4:51)

[SEÑALAR el diagrama. Recórrelo de arriba hacia abajo con el puntero.]

La solución es una arquitectura distribuida de tres niveles, alineada con los estándares industriales del Internet Industrial de las Cosas. La denominamos *MT-IoT*: Módulo de Transmisión IoT.

[SEÑALAR cada nivel.]

En el **Nivel de Borde** está la impresora fiscal HKA80, donde se origina la información crítica: los registros *Status S1* y el diagnóstico *SV2*, capturados por una interfaz síncrona transparente. En el **Nivel de Pasarela** está el Módulo de Transmisión IoT, basado en el ESP32: actúa como maestro de la interfaz local y como cliente seguro hacia la red. Y en el **Nivel Cloud** están el *broker* HiveMQ y el servidor Django, que gestionan la recepción de la telemetría y su visualización en tiempo real.

[**SEÑALAR las flechas.**]

El flujo de datos va de la impresora al ESP32 por el bus local; del ESP32 a la nube por Wi-Fi con TLS; y de la nube al dashboard por WebSockets seguros.

Diapositiva 6 — Nivel Edge: captura del *Status S1* (4:51 – 6:31)

[**SEÑALAR el detalle del bus síncrono.**]

El nivel de borde fue el más delicado, porque es donde se toca la información fiscal directamente.

Conviene aclarar una distinción importante. La impresora HKA80 se comunica con parámetros obligatorios del protocolo —9600 baudios, 8 bits de datos, paridad par y 1 bit de parada— sobre el estándar asíncrono RS-232. Ese fue el punto de partida. Sin embargo, el enlace definitivo de la solución **no es asíncrono, sino síncrono**: migramos la interconexión física local hacia una **interfaz serie síncrona dedicada operando a una frecuencia de reloj de 100 kHz**.

[**SEÑALAR el diagrama maestro-esclavo.**]

La diferencia es de fondo: el estándar asíncrono no comparte señal de reloj y el receptor debe sincronizarse por sí mismo, lo que lo hace vulnerable a la deriva de temporización. En la interfaz síncrona, el ESP32 genera el reloj como maestro y el periférico muestrea a ese compás, eliminando de raíz la deriva y garantizando una transmisión determinista.

[**SEÑALAR el detalle de la trama.**]

Para blindar las tramas tributarias validamos con redundancia longitudinal (LRC): operaciones XOR acumulativas contrastadas contra el *checksum* del periférico. Si la trama es válida, respondemos con un *ACK*; si no, con un *NAK*, reiniciando el bus para impedir que datos corruptos se propaguen.

Diapositiva 7 — Nivel Gateway: ESP32, ESP-IDF y kernel concurrente (6:31 – 8:07)

[**SEÑALAR el diagrama de los dos núcleos.**]

Para el nodo de transmisión seleccionamos la tarjeta ESP32 DevKitC V4, de doble núcleo: mientras un núcleo sondea la impresora cada 5 segundos, el segundo administra la pila de red y la mensajería MQTT. Así, las latencias de internet no bloquean la captura local de datos.

[**SEÑALAR el diagrama de exclusión mutua.**]

El corazón del diseño es el **kernel concurrente**. Sobre FreeRTOS, la tarea de adquisición fiscal y la tarea de red se ejecutan de forma independiente, y la coordinación entre ambas se resuelve con **exclusión mutua**: dos semáforos *mutex* de software que custodian los recursos compartidos del sistema.

- El primer *mutex*, `mqtt_data_mutex`, protege la **estructura de telemetría compartida**. Garantiza que la tarea de red lea siempre un conjunto de campos coherente y completo, y nunca una lectura a medio actualizar.

- El segundo, `bus_local_mutex`, arbitra el **acceso al bus físico**. Asegura que una sola tarea energice el canal de comunicación a la vez —ya sea el sondeo periódico o una actualización OTA—, preservando la integridad de cada transacción.

Diapositiva 8 — Nivel Cloud: MQTT v5.0 / TLS y Django + HTMX (8:07 – 9:46)

[**SEÑALAR el esquema de tópicos MQTT.**]

En la capa de red adoptamos el estándar **MQTT v5.0 sobre TLS 1.2**, hacia la infraestructura de HiveMQ Cloud. Se optó por MQTT por su mínima sobrecarga de cabecera y su modelo publicación-suscripción, y por la versión 5.0 por dos capacidades clave:

- Las ***User Properties***: metadatos en la cabecera del protocolo —modelo, versión de firmware, serial y RSSI— que el servidor lee sin deserializar el cuerpo del mensaje.

- El ***Last Will and Testament***: una "última voluntad"que el dispositivo registra al conectar; si pierde la conexión abruptamente, el *broker* la publica y el sistema sabe que el equipo cayó.

La seguridad usa el puerto 8883 sobre SSL y un *bundle* de certificados que valida la cadena de confianza.

[**SEÑALAR el diagrama del servidor.**]

En la nube construimos un ecosistema web con Django y HTMX. La pieza clave es un *worker* MQTT independiente que corre en segundo plano y descompone cada trama en eventos atómicos: cambios de estado, cortes Z, facturas, reconexiones y actualizaciones. Para la interfaz integramos el cliente MQTT de JavaScript directamente en el navegador, sobre el canal seguro del puerto 8884, y con HTMX logramos una experiencia de página única.

Diapositiva 9 — Justificación tecnológica (9:46 – 10:50)

[**SEÑALAR la tabla comparativa. Sé breve.**]

Permítannos justificar las dos decisiones tecnológicas más importantes.

Arduino frente a ESP-IDF. El modelo de ejecución de Arduino es un bucle secuencial mono-hilo; un `delay()` bloqueante en la pila Wi-Fi arriesgaba sobrepasar la ventana temporal del firmware legal del equipo. Por eso optamos por el framework nativo **ESP-IDF v5.5.2**, que nos dio gestión explícita de memoria en C y asignación determinista de tareas con FreeRTOS.

HTTP frente a MQTT. Una arquitectura HTTP REST tradicional tiene una sobrecarga de cabecera enorme para un mensaje que cambia cada 5 segundos. MQTT traslada los metadatos a la cabecera y aligera el cuerpo; más adelante les mostraremos la medición cuantitativa de ese ahorro.

[**TRANSICIÓN al Bloque 3**] Y con esto llegamos al bloque central: la validación y los resultados experimentales.

BLOQUE 3 — Validación y Resultados Experimentales

Tiempo estimado: 11,9 min (acumulado 10:50 – 22:42) · Diapositivas 10 a 18

> **Nota de enfoque:** este bloque presenta **qué se obtuvo y cómo se comprobó**, no los problemas del camino. Esos hallazgos ya están en el informe; aquí solo se mencionan si son indispensables para entender un resultado.

Diapositiva 10 — Metodología de validación: banco HIL (10:50 – 12:17)

[**SEÑALAR la foto del banco de pruebas.**]

Antes de los resultados, explicamos brevemente **cómo se validó el sistema**.

La validación de un sistema fiscal **no admite ensayos destructivos sobre equipos en producción**: inducir cortes de energía, corromper tramas o abortar una reescritura sobre una impresora real comprometería registros tributarios irreversibles.

Para conciliar rigor experimental y seguridad, construimos un entorno **Hardware-in-the-Loop** con dos niveles de emulación. El **primero** es un emulador en software, `impresora.py`, desarrollado con `pyserial` y los mismos parámetros eléctricos del enlace fiscal: implementa la máquina de estados de recepción, calcula el LRC y responde con las tramas S1, SV2 y los

acuses *ACK* y *NAK*. El **segundo** eleva la fidelidad: un ESP32 esclavo que emula íntegramente al periférico sobre el mismo bus y replica el protocolo completo de reescritura remota.

Contra ese banco validamos la OTA de extremo a extremo e inyectamos fallos de forma determinista —paquetes fuera de secuencia, *checksums* alterados, cortes de enlace— sin comprometer un equipo real.

[**PAUSA**] Todos los resultados que verán fueron medidos sobre este banco de pruebas.

Diapositiva 11 — Resultado 1: captura íntegra e integridad de la telemetría (12:17 – 13:51)

[**SEÑALAR la captura de la consola serial.**]

El primer resultado es la **captura íntegra y transparente del registro *Status S1***. Como ven en la traza, el módulo aísla los registros de la Memoria de Trabajo del equipo fiscal: el RIF del contribuyente, el número de registro de la máquina y los acumuladores de ventas diarias, y confirma el estado mecánico y lógico —modo fiscal y en espera, estado `0x60`, sin errores—.

Es transparente porque la interrogación se ejecuta desde una tarea dedicada de FreeRTOS, desacoplada de la tarea de red: el sondeo cada 5 segundos **no altera la máquina de estados del periférico ni interrumpe las transacciones en curso**. Y el análisis se hace sobre memoria estática, sin reservas dinámicas en la ruta crítica, lo que mantiene un tiempo de proceso acotado y constante.

En la misma línea, el segundo resultado: **la telemetría que llega a la nube es siempre válida y deserializable**. El campo de versión del periférico se entrega íntegro —`020507GD00`— y el *worker* de Django sanea los caracteres de control antes de persistir, de modo que ningún residuo llega a la base de datos.

Diapositiva 12 — Resultado 2: robustez e inmunidad del bus local (13:51 – 15:06)

[**SEÑALAR la tabla de los tres niveles.**]

El segundo resultado es la **robustez del bus local**, el eslabón físicamente más expuesto del sistema. La obtuvimos con una defensa escalonada en tres niveles:

- **Defensa temporal:** cada operación se ejecuta bajo ventanas estrictas —300 milisegundos para el *ACK* de reescritura, 2000 para las respuestas de estado y 3000 para el reporte S1— con una regla de máximo 3 reintentos.
- **Sanitización e integridad:** limpieza determinista de los búferes antes de cada transacción y validación LRC, que descarta cualquier trama alterada y responde con *NAK*.
- **Reconstrucción estructural:** si los errores superan el umbral, el firmware **desmonta y reancla por completo el bus local**, sin reiniciar el microcontrolador.

[**SEÑALAR el resultado final.**]

El resultado medido: ante la desconexión física del periférico o la inyección de tramas corruptas, el sistema **se recuperó automáticamente**, sin reiniciar el SoC y **sin que una sola trama inválida alcanzara la base de datos**.

Diapositiva 13 — Resultado 3: eficiencia de red, ahorro del 80,2 % (15:06 – 15:57)

[**SEÑALAR la tabla comparativa. Dila con calma.**]

El tercer resultado es la **eficiencia de red**. Medimos el consumo por cada ciclo de telemetría frente a una arquitectura HTTP tradicional.

[**SEÑALAR fila por fila.**]

- **Cabecera:** de unos 450 bytes en HTTP a 12 bytes en MQTT. Una reducción del **97,33 %**.

- **Cuerpo del mensaje:** de 328 a 142 bytes. Un **56,70 %** menos.

- **Consumo total por trama:** de **778 a 154 bytes**. Una reducción global del **80,2 %**.

[**PAUSA.**] Para un parque de impresoras que reporta cada 5 segundos durante todo el día, ese 80 % se traduce directamente en un ahorro del consumo de datos del contribuyente.

Diapositiva 14 — Resultado 4: resiliencia, cero pérdida de datos (15:57 – 17:18)

[**SEÑALAR la captura de *Log Capture* y el diagrama de flujo.**]

El cuarto resultado es la **resiliencia**: el sistema tolera fallas de energía y de red **sin pérdida de datos**.

Cuando el enlace se pierde, al expirar el *Keep-Alive*, el *broker* publica la "última voluntad"del dispositivo. Como ven en la captura, el *worker* interceptó la alerta `offline` y registró el evento como "Desconexión Abrupta", **congelando el último estado válido** sin alterar el histórico. El ciclo completo es: pérdida del enlace, reintentos con *backoff* exponencial, publicación del testamento, registro del evento y restauración del túnel.

Esto se apoya en una propiedad de diseño: el módulo es un **observador de estado sin memoria tributaria propia**. Toda la información fiscal reside en el periférico y se re-consulta en cada sondeo; el *gateway* no es una fuente de verdad que pueda corromperse.

[**SEÑALAR la tabla de escenarios.**]

En los **cuatro escenarios de falla** que evaluamos —corte abrupto de energía, micro-corte Wi-Fi, caída del *broker* y desconexión del periférico— **la pérdida de datos fue nula**.

Diapositiva 15 — Resultado 5: latencia End-to-End (T_{E2E}) (17:18 – 18:38)

[**SEÑALAR la tabla de latencias.**]

El quinto resultado es el **desempeño temporal**. Definimos la latencia de extremo a extremo como la suma de cinco etapas, desde el sondeo físico hasta el render en el navegador. Sobre **500 iteraciones continuas**, obtuvimos:

[**SEÑALAR cada fila.**]

- Sondeo síncrono del S1: **118 milisegundos** (dentro de la ventana fiscal).

- Empaquetado JSON: **2 milisegundos**.

- *Handshake* TLS 1.2: **45 milisegundos**.

- Publicación y tránsito por el *broker*: **85 milisegundos**, la etapa más variable.

- Renderizado del DOM: **18 milisegundos**.

La **latencia total promedio es de 268 milisegundos, con desviación de 15,3**: apenas el **5,36 % de la ventana de sondeo** de 5 segundos. El sistema opera en tiempo real estricto sin acumular retrasos. El análisis de varianza mostró que el **86,8 % de la dispersión** proviene de la etapa de red, es decir, del enlace del proveedor de Internet: la incertidumbre no está en el diseño embebido, sino en un factor externo y acotado.

Diapositiva 16 — Resultado 6: OTA de doble objetivo (18:38 – 20:09)

[SEÑALAR el diagrama de flujo de la OTA.]

El sexto resultado es la **actualización remota de firmware**, que fue además el aporte más disruptivo del trabajo: un protocolo OTA de **doble objetivo**.

[SEÑALAR la rama izquierda.]

El primer objetivo es el **gateway** ESP32. Al recibir la orden, activa una bandera que detiene el sondeo y escribe en la **partición OTA inactiva** de un esquema dual: si ocurre un corte de energía, el *bootloader* arranca desde la partición intacta.

[SEÑALAR la rama derecha.]

El segundo es el más complejo: **reescribir in situ el firmware del periférico fiscal**, que no tiene pila de red. El módulo actúa como *host* de programación intermedio: como el binario supera el *heap* libre del ESP32, la descarga se procesa por un **búfer circular de 1024 bytes**, en fragmentos con redundancia y hasta 3 reintentos con *ACK*. El cierre **solo se emite cuando los bytes confirmados igualan la longitud declarada**, así que una transferencia parcial nunca se compromete; y ante cortes de red, la reanudación usa **cabeceras HTTP Range**, sin reiniciar desde cero.

[SEÑALAR la captura de éxito.] Ambos objetivos quedaron validados de extremo a extremo.

Diapositiva 17 — Resultado 7: rendimiento global (20:09 – 21:31)

[SEÑALAR la tabla de perfilado de memoria.]

El séptimo resultado es el **rendimiento global**. Empecemos por la memoria, el recurso más restringido. Tras la reingeniería mediante `menuconfig` —desactivando la aceleración por IRAM de la pila Wi-Fi—, redujimos la **IRAM del 78,39 % al 52,26 %**: un ahorro de 34.256 bytes, un **26,13 % menos**, que fue justo lo que hizo viable reservar el *heap* para las operaciones de actualización.

[SEÑALAR la tabla del subclocking.]

En el plano energético adoptamos un **subclocking a 80 MHz** frente a los 240 MHz nominales: como la potencia dinámica es proporcional a la frecuencia, cae a **un tercio**, y con ella el salto térmico, así que el sistema opera **más frío**. Lo habilita el **acelerador AES por hardware**: los 80 MHz sostienen el *handshake* TLS en 45 milisegundos.

[SEÑALAR la tabla de SQLite.]

Y en la nube, la trazabilidad: con **10.000 registros**, la búsqueda indexada por serial se mantiene **por debajo de los 12 milisegundos**, y una depuración automática purga lo mayor a 90 días sin bloquear las consultas.

Diapositiva 18 — Resultado 8: comparativa estado previo vs. actual (21:31 – 22:42)

[SEÑALAR la tabla .ªntes y después". Cierra el bloque de resultados.]

Para sintetizar los resultados, la comparativa.

[SEÑALAR columna izquierda.]

Estado previo: el técnico conectaba físicamente una PC al puerto serial; observaba **un equipo a la vez**, sin visión global del parque; el diagnóstico era reactivo, posterior al fallo; no había historial de auditoría; y actualizar firmware exigía intervención en sitio.

[SEÑALAR columna derecha.]

Estado actual: monitoreo remoto y centralizado de **múltiples equipos en simultáneo**; detección temprana y automática de errores, con alertas; **bitácora de 90 días** consultable por serial; mantenimiento preventivo con datos reales; y actualización de firmware **desde la nube**, tanto del *gateway* como del periférico.

[**PAUSA**] En una frase: de un equipo aislado y observado a mano, a un parque conectado, trazable y administrable, con un 80,2 % menos de datos y 268 milisegundos de latencia de extremo a extremo.

[**TRANSICIÓN al Bloque 4**] Con esto concluimos los resultados. Pasamos al impacto y las conclusiones.

BLOQUE 4 — Impacto, Conclusiones y Escalabilidad

Tiempo estimado: 4,7 min (acumulado 22:42 – 27:21) · Diapositivas 19 a 22

Diapositiva 19 — Verificación de cumplimiento de objetivos (22:42 – 23:57)

[**SEÑALAR la matriz de objetivos del informe final.**]

Cerramos verificando el cumplimiento de objetivos. El informe incluye una **matriz de cumplimiento** que contrasta cada objetivo con su estado y su entregable técnico.

Los **ocho objetivos del plan se alcanzaron**, con un matiz que declaramos con transparencia:

- **Arquitectura, firmware, integración y documentación:** logrados.
- **Comunicación MQTT:** logrado. Cliente MQTT v5.0 sobre TLS 1.2, con *bundle* de certificados, *Last Will and Testament* y *User Properties*.
- **Servidor MQTT (recepción, almacenamiento y distribución): parcialmente logrado**. En lugar de desplegar un *broker* propio, se delegó el servicio al *broker* gestionado HiveMQ Cloud; la capa de recepción, almacenamiento y distribución sí se implementó.
- **Dashboard en tiempo real y validación:** logrados. Interfaz reactiva y métricas cuantificadas sobre banco HIL.

[**PAUSA**] Además, **ampliamos el alcance respecto al plan original** con un objetivo no previsto: el **módulo OTA de doble objetivo**, que resultó el componente más valioso del trabajo.

Diapositiva 20 — Aporte y beneficio para la organización (23:57 – 24:58)

[**SEÑALAR la infografía de beneficios.**]

¿Qué significa esto para The Factory HKA? Cuatro beneficios directos.

Primero, **visibilidad centralizada del parque**: lo que antes exigía recorrer físicamente cada equipo hoy se observa en un solo panel.

Segundo, **mantenimiento preventivo basado en datos**: la bitácora de 90 días permite analizar patrones de fallo y anticipar intervenciones, reduciendo tiempos de inactividad en los puntos de venta.

Tercero, **ahorro operativo en red**: el 80,2 % menos de datos por trama se traduce en menores costos de conectividad a escala de flota.

Y cuarto, **mantenimiento de firmware sin intervención en sitio**: la actualización remota permite corregir el software tanto del *gateway* como de la impresora desde la nube, algo clave para una empresa con equipos dispersos geográficamente.

Diapositiva 21 — Limitaciones y trabajo futuro (24:58 – 26:17)

[**SEÑALAR la lista. El jurado valora la autocrítica.**]
Toda solución tiene límites, y queremos ser claros con los de este trabajo.

- **Validación en laboratorio:** se validó sobre el banco HIL. Es un prototipo funcional y documentado, pero **aún no está desplegado en campo**; el siguiente paso es una prueba piloto con un conjunto reducido de contribuyentes.
- **Escala de la nube:** SQLite es adecuado para un piloto, pero para un parque nacional conviene migrar a un motor con mayor concurrencia, como PostgreSQL.
- **Seguridad y cumplimiento:** la información viaja cifrada, pero antes de un despliegue masivo debe revisarse el cumplimiento normativo sobre custodia de datos fiscales.
- **Dependencia de la red del contribuyente:** el sistema es resiliente, pero el 86,8 % de la dispersión proviene de ese factor externo.

Y como **trabajo futuro** proponemos: extender la telemetría a los reportes Z; incorporar analítica predictiva sobre el histórico de errores, para anticipar fallos mecánicos; y añadir aprovisionamiento masivo de dispositivos.

BLOQUE 5 — Cierre y Agradecimientos

Tiempo estimado: 1,6 min (acumulado 27:21 – 28:57) · Diapositiva 22

Diapositiva 23 — Cierre, agradecimientos y preguntas (27:21 – 28:57)

[**Diapositiva final. Baja el ritmo, cierra con seguridad.**]
[**PAUSA**] En conjunto, la solución deja las condiciones técnicas para que The Factory HKA centralice la telemetría de un parque nacional y transforme la gestión reactiva del mantenimiento en una práctica basada en datos.
[**PAUSA. Mira al jurado.**]
Quiero agradecer a la Universidad Simón Bolívar y a The Factory HKA; al profesor Miguel Díaz por su tutoría académica; a los ingenieros Pablo Arguinzones y Luis Muñoz por su acompañamiento industrial; y a todas las personas que me apoyaron durante estos meses.
Con esto damos por concluida la presentación y quedamos atentos a sus preguntas. Muchas gracias.
[**PAUSA. Espera. No agregues nada más.**]

Anexo — Preparación para la ronda de preguntas

Respuestas de 3 a 5 frases. Se dan en primera persona del plural, salvo si la pregunta es personal.

1. ¿Qué bus síncrono usaron: SPI o I2C? SPI. Fue lo que ofrecía el hardware de la impresora fiscal: el periférico expone su interfaz síncrona a través de su circuito integrado interno. No fue una decisión de comparar SPI contra I2C, sino la interfaz que el equipo ya traía; y encajó bien porque el ESP32 integra controlador SPI nativo y el esquema maestro-esclavo es directo.

2. ¿Por qué MQTT y no HTTP o WebSockets puros? Porque era el protocolo definido en el alcance de la propuesta y, además, resultó mediblemente más eficiente: redujo el volumen por trama en un 80,2 % frente a HTTP POST. Su modelo publicación-suscripción desacopla el dispositivo de los consumidores —el dashboard y el *worker* de ingesta reciben la misma telemetría—, y el *Last Will and Testament* permite detectar caídas abruptas. El margen de IRAM no dependió del protocolo: la presión venía de la pila Wi-Fi y de TLS, y se resolvió con `menuconfig`.

3. ¿Cómo garantizan que no se corrompe la información fiscal? Porque el ESP32 solo escucha: no manipula ni escribe la memoria fiscal, únicamente lee el *Status S1* y el *SV2*. No se puede corromper lo que no se modifica. Y en la actualización de firmware, antes de aplicar cualquier cambio se descarga y se almacena el binario de forma segura desde el portal.

4. ¿Qué pasa si se corta la energía a mitad de una OTA? En el periférico fiscal —el caso validado— el cierre de la transacción solo se emite cuando los bytes confirmados igualan la longitud declarada: una transferencia parcial nunca se compromete y una nueva orden reinicia el proceso desde cero. En el *gateway*, por diseño, la escritura ocurre sobre la partición OTA inactiva y el *bootloader* arrancaría desde la partición intacta; ese comportamiento está pendiente de confirmar en pruebas.

5. ¿Por qué SQLite y no PostgreSQL? SQLite es suficiente para el piloto: con 10.000 registros, la búsqueda indexada se mantiene por debajo de 12 milisegundos. Reconocemos que para un parque nacional conviene PostgreSQL por concurrencia; lo dejamos como trabajo futuro.

6. ¿Por qué ESP32 y no una Raspberry Pi o un PLC? Porque es un proyecto IoT, y para este caso el ESP32 es la plataforma indicada: doble núcleo con FreeRTOS, aceleración AES por hardware, bajo consumo y bajo costo, sin sistema operativo de propósito general. Además, es la plataforma que la empresa definió para sus equipos fiscales, lo que da continuidad con su ecosistema. Una Raspberry Pi añadiría un SO completo y mayor superficie de ataque, sin aportar nada que el sistema requiera.

7. ¿Es seguro enviar datos fiscales a la nube? Sí. El transporte usa TLS 1.2 de extremo a extremo, que cifra la información en tránsito, con validación de la cadena de confianza mediante *bundle* de certificados. Eso protege los datos frente a interceptación o manipulación.

8. ¿Cómo comprobaron que el problema del bus era por interferencia electromagnética? Lo evidenciamos por sus síntomas: sobre el enlace asíncrono aparecían errores de enmarcado y fallos en la verificación LRC, que desaparecieron al migrar a la interfaz síncrona con reloj dedicado. No lo caracterizamos con osciloscopio, y lo reconocemos como mejora: una campaña con instrumentación permitiría cuantificar la inmunidad del enlace.

9. ¿Cómo escala a miles de dispositivos? La arquitectura escala por diseño: añadir un dispositivo es añadir una suscripción a un tópico. El *broker* gestiona las sesiones y el *worker* de ingesta se desacopla del servidor web. Los cuellos de botella previsibles serían la base de datos y el aprovisionamiento, atacados con la retención de 90 días.

10. ¿La latencia de 268 ms es aceptable? Sí, con holgura: representa el 5,36 % del intervalo de sondeo de 5 segundos. Además, el 86,8 % de la dispersión proviene del enlace del proveedor de Internet, no del diseño embebido, lo que significa que en un enlace estable el sistema sería aún más regular.

11. ¿Por qué el objetivo del servidor MQTT quedó parcialmente logrado? Porque el plan contemplaba un *broker* propio, y en la arquitectura final se delegó al *broker* gestionado HiveMQ Cloud, que ofrece disponibilidad y seguridad sin administración local. La capa que sí implementamos fue la de recepción, almacenamiento y distribución en el servidor Django. Preferimos declararlo así en lugar de forzar la lectura del objetivo.

12. ¿Cuál fue el aporte más significativo? El puente síncrono heterogéneo de reprogramación inalámbrica. Es la capacidad más disruptiva porque traslada el mantenimiento de firmware a la nube y reduce la intervención física en campo, algo crítico para una empresa con equipos dispersos geográficamente.

Consejos de exposición

- **Ensaya en voz alta con cronómetro.** Si te pasas de 32 minutos, recorta la diapositiva 9 (justificación tecnológica) y la 17 (rendimiento global), que son las más prescindibles.
- **No leas las diapositivas.** Ellas llevan cifras y diagramas; tu voz cuenta la historia que las conecta.
- **Ancla las cifras fuertes** (80,2 %; 78,39 % $\rightarrow$ 52,26 %; 268 ms; cero pérdida de datos) diciéndolas despacio y con una pausa después.
- **Mantén el plural de forma consistente** en la parte técnica. Si se te escapa un τo", sigue adelante sin corregirte en voz alta.
- **Cuando señales un diagrama, recórrelo con el puntero** y nombra cada nivel antes de explicar su función.
- **Reserva 30 segundos de margen** antes de la primera pregunta.
- **Ten a mano una copia impresa** del informe con la matriz de objetivos y las tablas de resultados.